



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 04

NOMBRE COMPLETO: Antonio Manuel Zamora Ayala

N° de Cuenta: 320152896

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

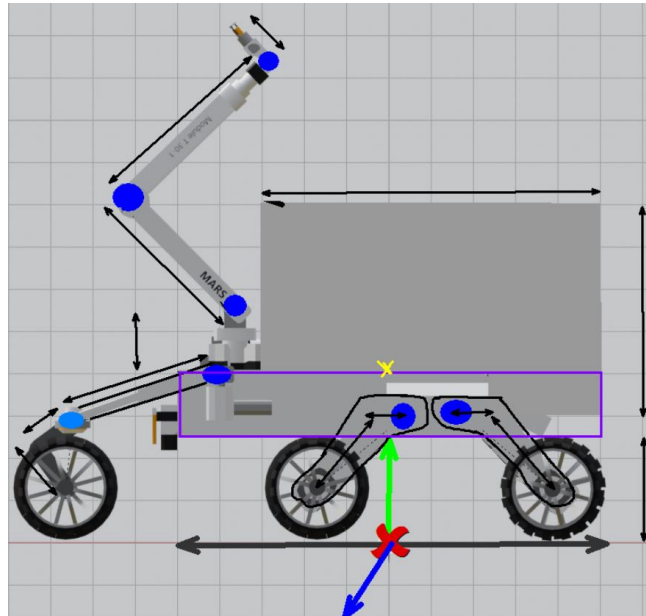
FECHA DE ENTREGA LÍMITE: 19 de septiembre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1. Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

- 1.1 Terminar el rover con: cabina, base, brazo con pinza con teclado (se puede girar cada articulación) y 6 llantas con teclado (se pueden girar cada una de las 6 llantas por separado).



Para este ejercicio modifiqué 3 archivos: *P04_320152896.cpp*, *Window.cpp* y *Window.h*.

En *P04_320152896.cpp*, implementé correctamente las llantas delanteras (me había faltado que el primer brazo también fuera en forma de "L") y creé las 4 llantas faltantes (en medio cerca, atrás cerca. En medio lejos, atrás lejos). Cada llanta está constituida por 2 ejes y un brazo de con un ángulo obtuso (un poco más abierto que el brazo en L).

Cada llanta está formada por los siguientes elementos: el primer eje está formado por un cubo y una esfera; el brazo de ángulo obtuso está formado por dos prismas cuadrangulares escalados y rotados de tal manera que formen dicha figura; y finalmente el último eje son dos cilindros, dónde uno pequeño sirve para ser la conexión entre el brazo y la llanta, y el último es la llanta.

Los bloques de código modificados son los siguientes:

```

520 // brazo 1 de la llanta delantera cercana
521 model = modelaux;
522 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.65f));
523 modelaux = model;
524 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
525 model = glm::scale(model, glm::vec3(0.4f, 1.0f, 0.4f));
526 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
527 color = glm::vec3(0.6f, 0.6f, 0.6f);
528 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
529 meshList[2]->RenderMeshGeometry();
530
531 model = modelaux;
532 model = glm::translate(model, glm::vec3(-2.25f, -0.5f, 0.0f));
533 model = glm::rotate(model, glm::radians(105.0f), glm::vec3(0.0f, 0.0f, 1.0f));
534 modelaux = model;
535 model = glm::scale(model, glm::vec3(0.4f, 4.0f, 0.4f));
536 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
537 color = glm::vec3(0.6f, 0.6f, 0.6f);
538 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
539 meshList[2]->RenderMeshGeometry();

```

```

618 // brazo 1 de la llanta delantera lejana
619 model = modelaux;
620 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -0.65f));
621 modelaux = model;
622 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
623 model = glm::scale(model, glm::vec3(0.4f, 1.0f, 0.4f));
624 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
625 color = glm::vec3(0.6f, 0.6f, 0.6f);
626 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
627 meshList[2]->RenderMeshGeometry();
628
629 model = modelaux;
630 model = glm::translate(model, glm::vec3(-2.25f, -0.5f, 0.0f));
631 model = glm::rotate(model, glm::radians(105.0f), glm::vec3(0.0f, 0.0f, 1.0f));
632 modelaux = model;
633 model = glm::scale(model, glm::vec3(0.4f, 4.0f, 0.4f));
634 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
635 color = glm::vec3(0.6f, 0.6f, 0.6f);
636 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
637 meshList[2]->RenderMeshGeometry();

```

```

665 //BASE PARA LA LLANTA CERCANA DE EN MEDIO
666 model = modelaux2;
667 model = glm::translate(model, glm::vec3(0.5f, -0.25f, 4.35f));
668 modelaux = model;
669 model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
670 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
671 color = glm::vec3(0.2f, 0.2f, 0.2f);
672 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
673 meshList[0]->RenderMesh();
674
675 // eje1 de la llanta cercana de enmedio (brazoL-base)
676 model = modelaux;
677 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.6f));
678 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion10()), glm::vec3(0.0f, 0.0f, 1.0f));
679 //model = glm::rotate(model, glm::radians(34.0f), glm::vec3(0.0f, 0.0f, 1.0f));
680 modelaux = model;
681 model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
682 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
683 color = glm::vec3(0.0f, 0.0f, 1.0f);
684 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
685 sp.render();
686
687 //brazo en L de la llanta cercana de enmedio
688 model = modelaux;
689 model = glm::translate(model, glm::vec3(-0.65f, 0.0f, 0.6f));
690 modelaux = model;
691 model = glm::scale(model, glm::vec3(2.0f, 1.0f, 0.7f));
692 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
693 color = glm::vec3(0.6f, 0.6f, 0.6f);
694 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
695 meshList[0]->RenderMesh();
696
697 model = modelaux;
698 model = glm::translate(model, glm::vec3(-1.92f, -1.63f, 0.0f));
699 model = glm::rotate(model, glm::radians(55.0f), glm::vec3(0.0f, 0.0f, 1.0f));
700 modelaux = model;
701 model = glm::scale(model, glm::vec3(4.5f, 1.0f, 0.7f));
702 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
703 color = glm::vec3(0.6f, 0.6f, 0.6f);
704 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
705 meshList[0]->RenderMesh();
706
707 model = modelaux;
708 model = glm::translate(model, glm::vec3(-1.75f, 0.0f, 0.6f));
709 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
710 modelaux = model;
711 model = glm::scale(model, glm::vec3(0.4f, 0.5f, 0.4f));
712 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
713 color = glm::vec3(0.2f, 0.2f, 0.2f);
714 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
715 meshList[2]->RenderMeshGeometry();
716
717 //llanta cercana de enmedio
718 model = modelaux;
719 model = glm::translate(model, glm::vec3(0.0f, 0.5f, 0.0f));
720 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion11()), glm::vec3(0.0f, 1.0f, 0.0f));
721 modelaux = model;
722 model = glm::scale(model, glm::vec3(1.8f, 0.5f, 1.8f));
723 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
724 color = glm::vec3(0.0f, 0.0f, 0.0f);
725 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
726 meshList[2]->RenderMeshGeometry();

```

```

728 //BASE PARA LA LLANTA CERCANA DE ATRÁS
729 model = modelaux2;
730 model = glm::translate(model, glm::vec3(2.0f, -0.25f, 4.35f));
731 modelaux = model;
732 model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
733 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
734 color = glm::vec3(0.2f, 0.2f, 0.2f);
735 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
736 meshList[0]-->RenderMesh();
737
738 // ejel de la llanta cercana de atrás (brazoL-base)
739 model = modelaux;
740 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.6f));
741 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion12()), glm::vec3(0.0f, 0.0f, 1.0f));
742 modelaux = model;
743 model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
744 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
745 color = glm::vec3(0.0f, 0.0f, 1.0f);
746 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
747 sp.render();
748
749 //brazo en L de la llanta cercana de atrás
750 model = modelaux;
751 model = glm::translate(model, glm::vec3(0.65f, 0.0f, 0.6f));
752 modelaux = model;
753 model = glm::scale(model, glm::vec3(2.0f, 1.0f, 0.7f));
754 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
755 color = glm::vec3(0.6f, 0.6f, 0.6f);
756 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
757 meshList[0]-->RenderMesh();
758
759 model = modelaux;
760 model = glm::translate(model, glm::vec3(1.92f, -1.63f, 0.0f));
761 model = glm::rotate(model, glm::radians(-55.0f), glm::vec3(0.0f, 0.0f, 1.0f));
762 modelaux = model;
763 model = glm::scale(model, glm::vec3(4.5f, 1.0f, 0.7f));
764 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
765 color = glm::vec3(0.6f, 0.6f, 0.6f);
766 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
767 meshList[0]-->RenderMesh();
768
769 model = modelaux;
770 model = glm::translate(model, glm::vec3(1.75f, 0.0f, 0.6f));
771 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
772 modelaux = model;
773 model = glm::scale(model, glm::vec3(0.4f, 0.5f, 0.4f));
774 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
775 color = glm::vec3(0.2f, 0.2f, 0.2f);
776 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
777 meshList[2]-->RenderMeshGeometry();
778
779 //llanta cercana de atrás
780 model = modelaux;
781 model = glm::translate(model, glm::vec3(0.0f, 0.5f, 0.0f));
782 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion13()), glm::vec3(0.0f, 1.0f, 0.0f));
783 modelaux = model;
784 model = glm::scale(model, glm::vec3(1.8f, 0.5f, 1.8f));
785 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
786 color = glm::vec3(0.0f, 0.0f, 0.0f);
787 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
788 meshList[2]-->RenderMeshGeometry();

```

```

790 //BASE PARA LA LLANTA LEJANA DE EN MEDIO
791 model = modelaux2;
792 model = glm::translate(model, glm::vec3(0.5f, -0.25f, -4.35f));
793 modelaux = model;
794 model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
795 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
796 color = glm::vec3(0.2f, 0.2f, 0.2f);
797 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
798 meshList[0]-->RenderMesh();
799
800 // ejel de la llanta lejana de enmedio (brazoL-base)
801 model = modelaux;
802 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -0.6f));
803 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion14()), glm::vec3(0.0f, 0.0f, 1.0f));
804 //model = glm::rotate(model, glm::radians(34.0f), glm::vec3(0.0f, 0.0f, 1.0f));
805 modelaux = model;
806 model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
807 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
808 color = glm::vec3(0.0f, 0.0f, 1.0f);
809 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
810 sp.render();
811
812 //brazo en L de la llanta lejana de enmedio
813 model = modelaux;
814 model = glm::translate(model, glm::vec3(-0.65f, 0.0f, -0.6f));
815 modelaux = model;
816 model = glm::scale(model, glm::vec3(2.0f, 1.0f, 0.7f));
817 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
818 color = glm::vec3(0.6f, 0.6f, 0.6f);
819 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
820 meshList[0]-->RenderMesh();
821
822 model = modelaux;
823 model = glm::translate(model, glm::vec3(-1.92f, -1.63f, 0.0f));
824 model = glm::rotate(model, glm::radians(55.0f), glm::vec3(0.0f, 0.0f, 1.0f));
825 modelaux = model;
826 model = glm::scale(model, glm::vec3(4.5f, 1.0f, 0.7f));
827 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
828 color = glm::vec3(0.6f, 0.6f, 0.6f);
829 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
830 meshList[0]-->RenderMesh();
831
832 model = modelaux;
833 model = glm::translate(model, glm::vec3(-1.75f, 0.0f, -0.6f));
834 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
835 modelaux = model;
836 model = glm::scale(model, glm::vec3(0.4f, 0.5f, 0.4f));
837 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
838 color = glm::vec3(0.2f, 0.2f, 0.2f);
839 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
840 meshList[2]-->RenderMeshGeometry();
841
842 //llanta lejana de enmedio
843 model = modelaux;
844 model = glm::translate(model, glm::vec3(0.0f, -0.5f, 0.0f));
845 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion15()), glm::vec3(0.0f, 1.0f, 0.0f));
846 modelaux = model;
847 model = glm::scale(model, glm::vec3(1.8f, 0.5f, 1.8f));
848 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
849 color = glm::vec3(0.0f, 0.0f, 0.0f);
850 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
851 meshList[2]-->RenderMeshGeometry();

```

```

853 //BASE PARA LA LLANTA LEJANA DE ATRÁS
854 model = modelaux2;
855 model = glm::translate(model, glm::vec3(2.0f, -0.25f, -4.35f));
856 modelaux = model;
857 model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
858 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
859 color = glm::vec3(0.2f, 0.2f, 0.2f);
860 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
861 meshList[0]->RenderMesh();
862
863 // ejel de la llanta lejana de atrás (brazol-base)
864 model = modelaux;
865 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -0.6f));
866 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion16()), glm::vec3(0.0f, 0.0f, 1.0f));
867 modelaux = model;
868 model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
869 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
870 color = glm::vec3(0.0f, 0.0f, 1.0f);
871 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
872 sp.render();
873
874 //brazo en L de la llanta lejana de atrás
875 model = modelaux;
876 model = glm::translate(model, glm::vec3(0.65f, 0.0f, -0.6f));
877 modelaux = model;
878 model = glm::scale(model, glm::vec3(2.0f, 1.0f, 0.7f));
879 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
880 color = glm::vec3(0.6f, 0.6f, 0.6f);
881 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
882 meshList[0]->RenderMesh();
883
884 model = modelaux;
885 model = glm::translate(model, glm::vec3(1.92f, -1.63f, 0.0f));
886 model = glm::rotate(model, glm::radians(-55.0f), glm::vec3(0.0f, 0.0f, 1.0f));
887 modelaux = model;
888 model = glm::scale(model, glm::vec3(4.5f, 1.0f, 0.7f));
889 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
890 color = glm::vec3(0.6f, 0.6f, 0.6f);
891 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
892 meshList[0]->RenderMesh();
893
894 model = modelaux;
895 model = glm::translate(model, glm::vec3(1.75f, 0.0f, -0.6f));
896 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
897 modelaux = model;
898 model = glm::scale(model, glm::vec3(0.4f, 0.5f, 0.4f));
899 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
900 color = glm::vec3(0.2f, 0.2f, 0.2f);
901 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
902 meshList[2]->RenderMeshGeometry();
903
904 //llanta lejana de atrás
905 model = modelaux;
906 model = glm::translate(model, glm::vec3(0.0f, -0.5f, 0.0f));
907 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion17()), glm::vec3(0.0f, 1.0f, 0.0f));
908 modelaux = model;
909 model = glm::scale(model, glm::vec3(1.8f, 0.5f, 1.8f));
910 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
911 color = glm::vec3(0.0f, 0.0f, 0.0f);
912 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
913 meshList[2]->RenderMeshGeometry();

```

En *Window.cpp*, modifiqué los siguientes bloques de código:

```

27 articulation9 = 0.0f;
28 articulation10 = 0.0f;
29 articulation11 = 0.0f;
30 articulation12 = 0.0f;
31 articulation13 = 0.0f;
32 articulation14 = 0.0f;
33 articulation15 = 0.0f;
34 articulation16 = 0.0f;
35 articulation17 = 0.0f;

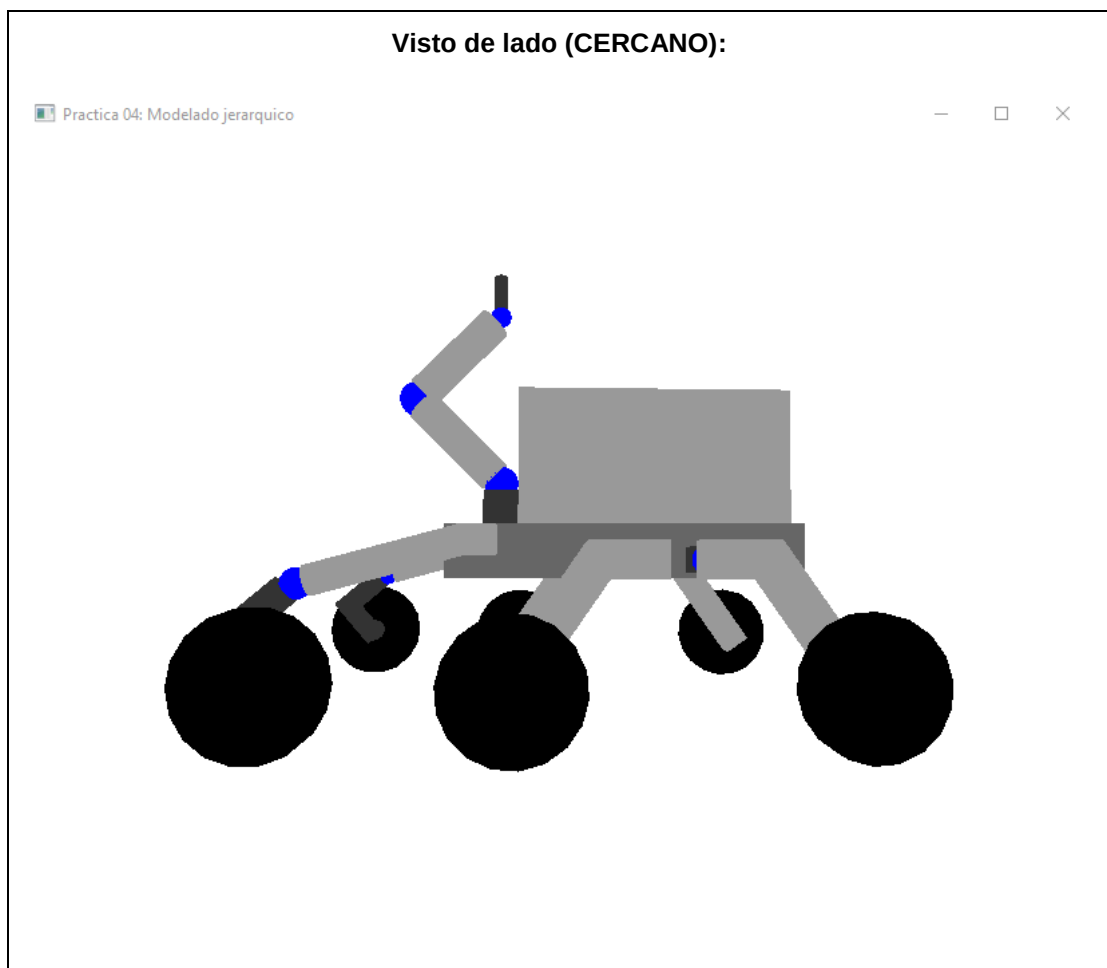
176 if (key == GLFW_KEY_U)
177 {
178     theWindow->articulation10 += 10.0;
179 }
180 if (key == GLFW_KEY_Y)
181 {
182     theWindow->articulation11 += 10.0;
183 }
184 if (key == GLFW_KEY_Z)
185 {
186     theWindow->articulation12 += 10.0;
187 }
188 if (key == GLFW_KEY_X)
189 {
190     theWindow->articulation13 += 10.0;
191 }
192 if (key == GLFW_KEY_C)
193 {
194     theWindow->articulation14 += 10.0;
195 }
196 if (key == GLFW_KEY_V)
197 {
198     theWindow->articulation15 += 10.0;
199 }
200 if (key == GLFW_KEY_B)
201 {
202     theWindow->articulation16 += 10.0;
203 }
204 if (key == GLFW_KEY_N)
205 {
206     theWindow->articulation17 += 10.0;
207 }

```

Finalmente, en *Window.h*, modifiqué los siguientes bloques de código:

```
31     GLfloat getarticulacion9() { return articulacion9; }
32     GLfloat getarticulacion10() { return articulacion10; }
33     GLfloat getarticulacion11() { return articulacion11; }
34     GLfloat getarticulacion12() { return articulacion12; }
35     GLfloat getarticulacion13() { return articulacion13; }
36     GLfloat getarticulacion14() { return articulacion14; }
37     GLfloat getarticulacion15() { return articulacion15; }
38     GLfloat getarticulacion16() { return articulacion16; }
39     GLfloat getarticulacion17() { return articulacion17; }
40
45     GLfloat rotax, rotay, rotaz, articulacion1, articulacion2, articulacion3,
46     articulacion4, articulacion5, articulacion6, articulacion7,
47     articulacion8, articulacion9, articulacion10, articulacion11,
48     articulacion12, articulacion13, articulacion14, articulacion15,
49     articulacion16, articulacion17;
```

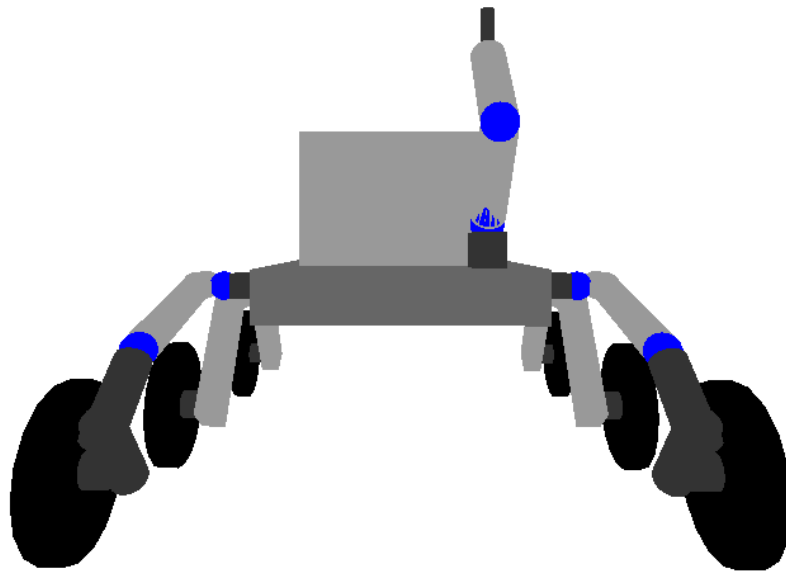
Adjunto capturas de pantalla de mi cohete:



Visto de frente:

Practica 04: Modelado jerarquico

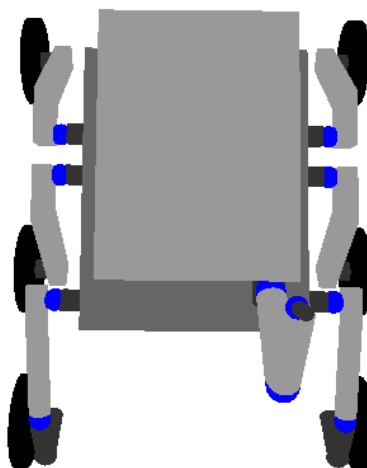
— □ ×

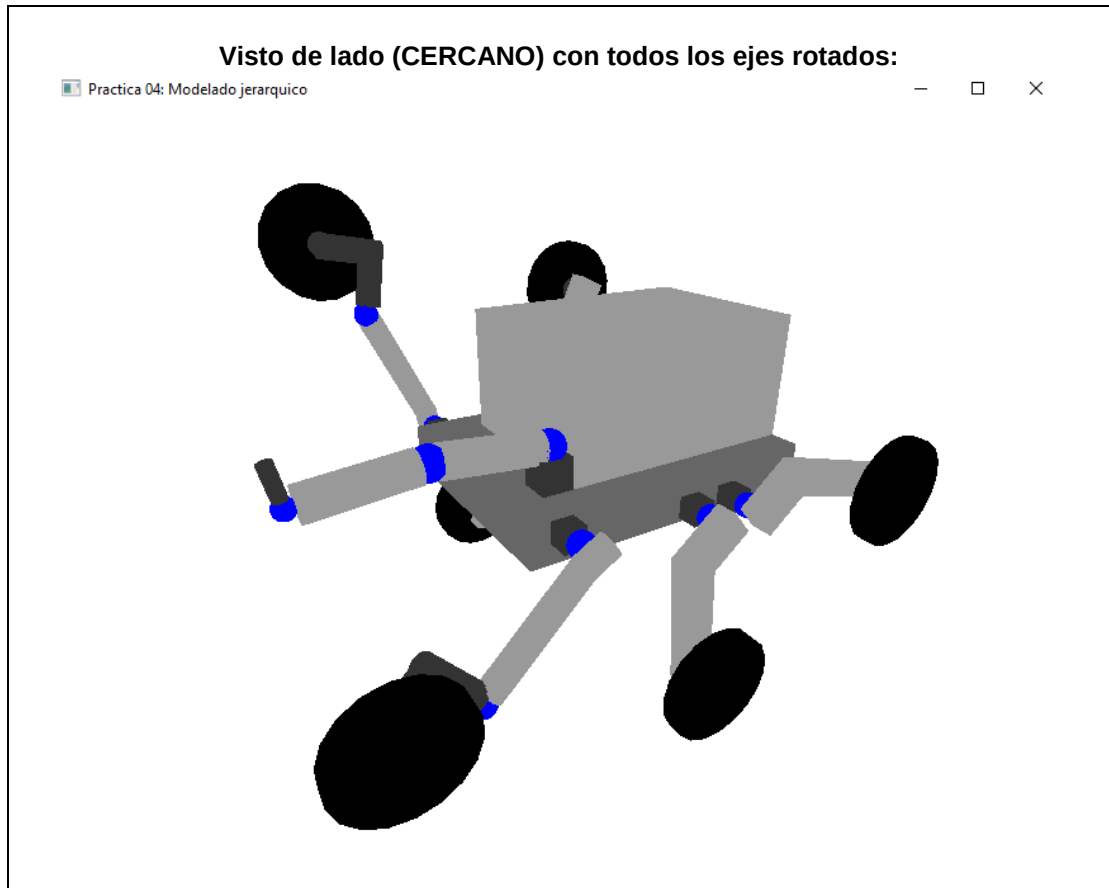


Visto de arriba:

Practica 04: Modelado jerarquico

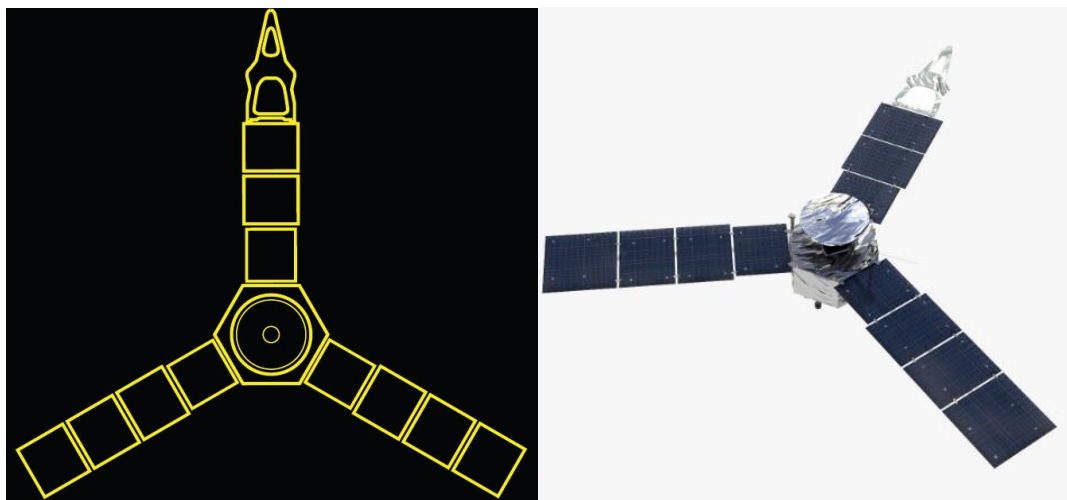
— □ ×





1.2 Crear Jerárquicamente una sonda espacial.

Elegí la sonda espacial Juno como referencia:



Para este ejercicio modifiqué 3 archivos: *P04_320152896.cpp*, *Window.cpp* y *Window.h*.

En *P04_320152896.cpp*, primero creé una nueva función para dibujar un prisma triangular que utilicé para crear el magnetómetro y dentro de *main()* la inicialicé.

```
276 // Prisma triangular unitario
277 void CrearPrismaTriangular()
278 {
279     unsigned int indices_prismaTriang[] = {
280         // Cara triangular frontal
281         0, 1, 2,
282         // Cara triangular trasera
283         3, 5, 4,
284         // Cara rectangular inferior
285         0, 3, 4,
286         4, 1, 0,
287         // Cara rectangular derecha
288         1, 4, 5,
289         5, 2, 1,
290         // Cara rectangular izquierda
291         2, 5, 3,
292         3, 0, 2
293     };
294     GLfloat vertices_prismaTriang[] = {
295         // Triángulo frontal
296         -0.5f, -0.5f, 0.5f, // 0
297         0.5f, -0.5f, 0.5f, // 1
298         0.0f, 0.5f, 0.5f, // 2
299         // Triángulo trasero
300         -0.5f, -0.5f, -0.5f, // 3
301         0.5f, -0.5f, -0.5f, // 4
302         0.0f, 0.5f, -0.5f, // 5
303     };
304     Mesh* prismaTriang = new Mesh();
305     prismaTriang->CreateMesh(vertices_prismaTriang, indices_prismaTriang, 18, 24);
306     meshList.push_back(prismaTriang);
307 }
340 CrearPrismaTriangular(); //índice 6 en MeshList
```

Después modifiqué los siguientes bloques de código para crear jerárquicamente mi sonda espacial. Primero creé dos *modelaux* nuevos, uno para la sonda en general y otro para la base de la sonda, ya que al rotar la sonda, todos los paneles deben rotar a la par.

```

372 glm::mat4 modelaux3(1.0); // para la sonda espacial
373 glm::mat4 modelaux4(1.0); // para la base de la sonda espacial

984 /*
985 colores "metal" y otras tonalidades para la sonda:
986 titanio: 0.60f, 0.63f, 0.66f
987 aluminio: 0.77f, 0.79f, 0.82f
988 acero inoxidable: 0.70f, 0.72f, 0.74f
989 azul de los paneles: 0.05f, 0.08f, 0.25f
990
991 --- Sonda espacial ---
992
993 La sonda se dividirá en: base, antena, ejeBase-Antena, paneles solares (11),
994 ejesBase-PanelSolar(3), magnetómetro, ejeMagnetómetro-PanelSolar,
995 ejesPanelSolar-PanelSolar (8)
996 */
997
998 // Antena y base
999 model = modelaux3;
1000 model = glm::translate(model, glm::vec3(-45.0f, 1.5f, 0.0f));
1001 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion18()), glm::vec3(0.0f, 1.0f, 0.0f));
1002 modelaux = model;
1003 model = glm::scale(model, glm::vec3(2.0f, 0.3f, 2.0f));
1004 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1005 color = glm::vec3(0.56f, 0.59f, 0.64f);
1006 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1007 sp.render();
1008
1009 model = modelaux;
1010 model = glm::translate(model, glm::vec3(0.0f, -0.5f, 0.0f));
1011 model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 1.0f, 0.0f));
1012 modelaux = model;
1013 model = glm::scale(model, glm::vec3(2.0f, 0.75f, 2.0f));
1014 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1015 color = glm::vec3(0.2f, 0.2f, 0.2f);
1016 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1017 meshList[0]->RenderMesh();
1018
1019 // Base (cilindro con resolución 6)
1020 model = modelaux;
1021 model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
1022 modelaux = model;
1023 modelaux4 = model;
1024 model = glm::scale(model, glm::vec3(3.0f, 2.5f, 3.0f));
1025 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1026 color = glm::vec3(0.60f, 0.63f, 0.66f);
1027 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1028 meshList[5]->RenderMeshGeometry();

```

```

1030 // PARA EL PRIMER CONJUNTO DE PANELES SOLARES
1031
1032 // Eje1 Base-Panel1
1033 model = modelaux4;
1034 model = glm::translate(model, glm::vec3(0.0f, -0.5f, 2.8f));
1035 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion19()), glm::vec3(1.0f, 0.0f, 0.0f));
1036 modelaux = model;
1037 model = glm::scale(model, glm::vec3(2.0f, 0.5f, 0.5f));
1038 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1039 color = glm::vec3(0.2f, 0.2f, 0.2f);
1040 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1041 meshList[0]->RenderMesh();
1042
1043 // Panel solar 1
1044 model = modelaux;
1045 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.65f));
1046 modelaux = model;
1047 model = glm::scale(model, glm::vec3(2.5f, 1.0f, 2.8f));
1048 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1049 color = glm::vec3(0.05f, 0.08f, 0.25f);
1050 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1051 meshList[0]->RenderMesh();
1052
1053 // Eje1 Panel1-Panel2
1054 model = modelaux;
1055 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.65f));
1056 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion20()), glm::vec3(1.0f, 0.0f, 0.0f));
1057 modelaux = model;
1058 model = glm::scale(model, glm::vec3(2.0f, 0.5f, 0.5f));
1059 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1060 color = glm::vec3(0.2f, 0.2f, 0.2f);
1061 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1062 meshList[0]->RenderMesh();
1063
1064 // Panel solar 2
1065 model = modelaux;
1066 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.65f));
1067 modelaux = model;
1068 model = glm::scale(model, glm::vec3(3.0f, 1.0f, 2.8f));
1069 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1070 color = glm::vec3(0.05f, 0.08f, 0.25f);
1071 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1072 meshList[0]->RenderMesh();
1073
1074 // Eje1 Panel2-Panel3
1075 model = modelaux;
1076 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.65f));
1077 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion21()), glm::vec3(1.0f, 0.0f, 0.0f));
1078 modelaux = model;
1079 model = glm::scale(model, glm::vec3(2.0f, 0.5f, 0.5f));
1080 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1081 color = glm::vec3(0.2f, 0.2f, 0.2f);
1082 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1083 meshList[0]->RenderMesh();

```

```

1085 // Panel solar 3
1086 model = modelaux;
1087 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.65f));
1088 modelaux = model;
1089 model = glm::scale(model, glm::vec3(3.0f, 1.0f, 2.8f));
1090 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1091 color = glm::vec3(0.05f, 0.08f, 0.25f);
1092 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1093 meshList[0]->RenderMesh();
1094
1095 // Eje1 Panel3-Panel4
1096 model = modelaux;
1097 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.65f));
1098 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion22()), glm::vec3(1.0f, 0.0f, 0.0f));
1099 modelaux = model;
1100 model = glm::scale(model, glm::vec3(2.0f, 0.5f, 0.5f));
1101 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1102 color = glm::vec3(0.2f, 0.2f, 0.2f);
1103 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1104 meshList[0]->RenderMesh();
1105
1106 // Panel solar 4
1107 model = modelaux;
1108 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.65f));
1109 modelaux = model;
1110 model = glm::scale(model, glm::vec3(3.0f, 1.0f, 2.8f));
1111 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1112 color = glm::vec3(0.05f, 0.08f, 0.25f);
1113 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1114 meshList[0]->RenderMesh();
1115
1116 // PARA EL SEGUNDO CONJUNTO DE PANELES SOLARES
1117
1118 // Eje1 Base-Panel1
1119 model = modelaux4;
1120 model = glm::translate(model, glm::vec3(2.4f, -0.5f, -1.45f));
1121 model = glm::rotate(model, glm::radians(-60.0f), glm::vec3(0.0f, 1.0f, 0.0f));
1122 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion23()), glm::vec3(-1.0f, 0.0f, 0.0f));
1123 modelaux = model;
1124 model = glm::scale(model, glm::vec3(2.0f, 0.5f, 0.5f));
1125 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1126 color = glm::vec3(0.2f, 0.2f, 0.2f);
1127 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1128 meshList[0]->RenderMesh();
1129
1130 // Panel solar 1
1131 model = modelaux;
1132 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.65f));
1133 modelaux = model;
1134 model = glm::scale(model, glm::vec3(2.5f, 1.0f, 2.8f));
1135 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1136 color = glm::vec3(0.05f, 0.08f, 0.25f);
1137 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1138 meshList[0]->RenderMesh();

```

```

1140 // Eje1 Panel1-Panel2
1141 model = modelaux;
1142 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.65f));
1143 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion24()), glm::vec3(-1.0f, 0.0f, 0.0f));
1144 modelaux = model;
1145 model = glm::scale(model, glm::vec3(2.0f, 0.5f, 0.5f));
1146 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1147 color = glm::vec3(0.2f, 0.2f, 0.2f);
1148 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1149 meshList[0]->RenderMesh();
1150
1151 // Panel solar 2
1152 model = modelaux;
1153 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.65f));
1154 modelaux = model;
1155 model = glm::scale(model, glm::vec3(3.0f, 1.0f, 2.8f));
1156 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1157 color = glm::vec3(0.05f, 0.08f, 0.25f);
1158 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1159 meshList[0]->RenderMesh();
1160
1161 // Eje1 Panel2-Panel3
1162 model = modelaux;
1163 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.65f));
1164 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion25()), glm::vec3(-1.0f, 0.0f, 0.0f));
1165 modelaux = model;
1166 model = glm::scale(model, glm::vec3(2.0f, 0.5f, 0.5f));
1167 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1168 color = glm::vec3(0.2f, 0.2f, 0.2f);
1169 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1170 meshList[0]->RenderMesh();
1171
1172 // Panel solar 3
1173 model = modelaux;
1174 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.65f));
1175 modelaux = model;
1176 model = glm::scale(model, glm::vec3(3.0f, 1.0f, 2.8f));
1177 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1178 color = glm::vec3(0.05f, 0.08f, 0.25f);
1179 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1180 meshList[0]->RenderMesh();
1181
1182 // Eje1 Panel3-Panel4
1183 model = modelaux;
1184 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.65f));
1185 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion26()), glm::vec3(-1.0f, 0.0f, 0.0f));
1186 modelaux = model;
1187 model = glm::scale(model, glm::vec3(2.0f, 0.5f, 0.5f));
1188 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1189 color = glm::vec3(0.2f, 0.2f, 0.2f);
1190 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1191 meshList[0]->RenderMesh();

```

```

1193 // Panel solar 4
1194 model = modelaux;
1195 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.65f));
1196 modelaux = model;
1197 model = glm::scale(model, glm::vec3(3.0f, 1.0f, 2.8f));
1198 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1199 color = glm::vec3(0.05f, 0.08f, 0.25f);
1200 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1201 meshList[0]->RenderMesh();
1202
1203 // PARA EL TERCER CONJUNTO DE PANELES SOLARES Y EL MAGNETÓMETRO
1204
1205 model = modelaux4;
1206 model = glm::translate(model, glm::vec3(-2.4f, -0.5f, -1.45f));
1207 model = glm::rotate(model, glm::radians(60.0f), glm::vec3(0.0f, 1.0f, 0.0f));
1208 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion27()), glm::vec3(-1.0f, 0.0f, 0.0f));
1209 modelaux = model;
1210 model = glm::scale(model, glm::vec3(2.0f, 0.5f, 0.5f));
1211 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1212 color = glm::vec3(0.2f, 0.2f, 0.2f);
1213 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1214 meshList[0]->RenderMesh();
1215
1216 // Panel solar 1
1217 model = modelaux;
1218 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.65f));
1219 modelaux = model;
1220 model = glm::scale(model, glm::vec3(2.5f, 1.0f, 2.8f));
1221 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1222 color = glm::vec3(0.05f, 0.08f, 0.25f);
1223 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1224 meshList[0]->RenderMesh();
1225
1226 // Eje1 Panel1-Panel2
1227 model = modelaux;
1228 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.65f));
1229 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion28()), glm::vec3(-1.0f, 0.0f, 0.0f));
1230 modelaux = model;
1231 model = glm::scale(model, glm::vec3(2.0f, 0.5f, 0.5f));
1232 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1233 color = glm::vec3(0.2f, 0.2f, 0.2f);
1234 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1235 meshList[0]->RenderMesh();
1236
1237 // Panel solar 2
1238 model = modelaux;
1239 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.65f));
1240 modelaux = model;
1241 model = glm::scale(model, glm::vec3(3.0f, 1.0f, 2.8f));
1242 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1243 color = glm::vec3(0.05f, 0.08f, 0.25f);
1244 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1245 meshList[0]->RenderMesh();

```

```

1247 // Eje1 Panel2-Panel3
1248 model = modelaux;
1249 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.65f));
1250 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion29()), glm::vec3(-1.0f, 0.0f, 0.0f));
1251 modelaux = model;
1252 model = glm::scale(model, glm::vec3(2.0f, 0.5f, 0.5f));
1253 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1254 color = glm::vec3(0.2f, 0.2f, 0.2f);
1255 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1256 meshList[0]->RenderMesh();
1257
1258 // Panel solar 3
1259 model = modelaux;
1260 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.65f));
1261 modelaux = model;
1262 model = glm::scale(model, glm::vec3(3.0f, 1.0f, 2.8f));
1263 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1264 color = glm::vec3(0.05f, 0.08f, 0.25f);
1265 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1266 meshList[0]->RenderMesh();
1267
1268 // Eje1 Panel3-Magnetometro
1269 model = modelaux;
1270 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.65f));
1271 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion30()), glm::vec3(-1.0f, 0.0f, 0.0f));
1272 modelaux = model;
1273 model = glm::scale(model, glm::vec3(2.0f, 0.5f, 0.5f));
1274 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1275 color = glm::vec3(0.2f, 0.2f, 0.2f);
1276 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1277 meshList[0]->RenderMesh();
1278
1279 //Magnetómetro
1280 model = modelaux;
1281 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -2.65f));
1282 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(-1.0f, 0.0f, 0.0f));
1283 modelaux = model;
1284 model = glm::scale(model, glm::vec3(2.8f, 4.8f, 0.5f));
1285 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1286 color = glm::vec3(0.56f, 0.59f, 0.64f);
1287 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1288 meshList[6]->RenderMesh();
1289
1290 model = modelaux;
1291 model = glm::translate(model, glm::vec3(0.0f, -0.25f, 0.0f));
1292 modelaux = model;
1293 model = glm::scale(model, glm::vec3(1.75f, 3.25f, 0.503f));
1294 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1295 color = glm::vec3(1.0f, 1.0f, 1.0f);
1296 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1297 meshList[6]->RenderMesh();
1298
1299 model = modelaux;
1300 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
1301 modelaux = model;
1302 model = glm::scale(model, glm::vec3(1.15f, 0.5f, 0.505f));
1303 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1304 color = glm::vec3(0.56f, 0.59f, 0.64f);
1305 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1306 meshList[0]->RenderMesh();

```

En *Window.cpp*, modifiqué los siguientes bloques de código:

```
36 articulation18 = 0.0f;
37 articulation19 = 0.0f;
38 articulation20 = 0.0f;
39 articulation21 = 0.0f;
40 articulation22 = 0.0f;
41 articulation23 = 0.0f;
42 articulation24 = 0.0f;
43 articulation25 = 0.0f;
44 articulation26 = 0.0f;
45 articulation27 = 0.0f;
46 articulation28 = 0.0f;
47 articulation29 = 0.0f;
48 articulation30 = 0.0f;

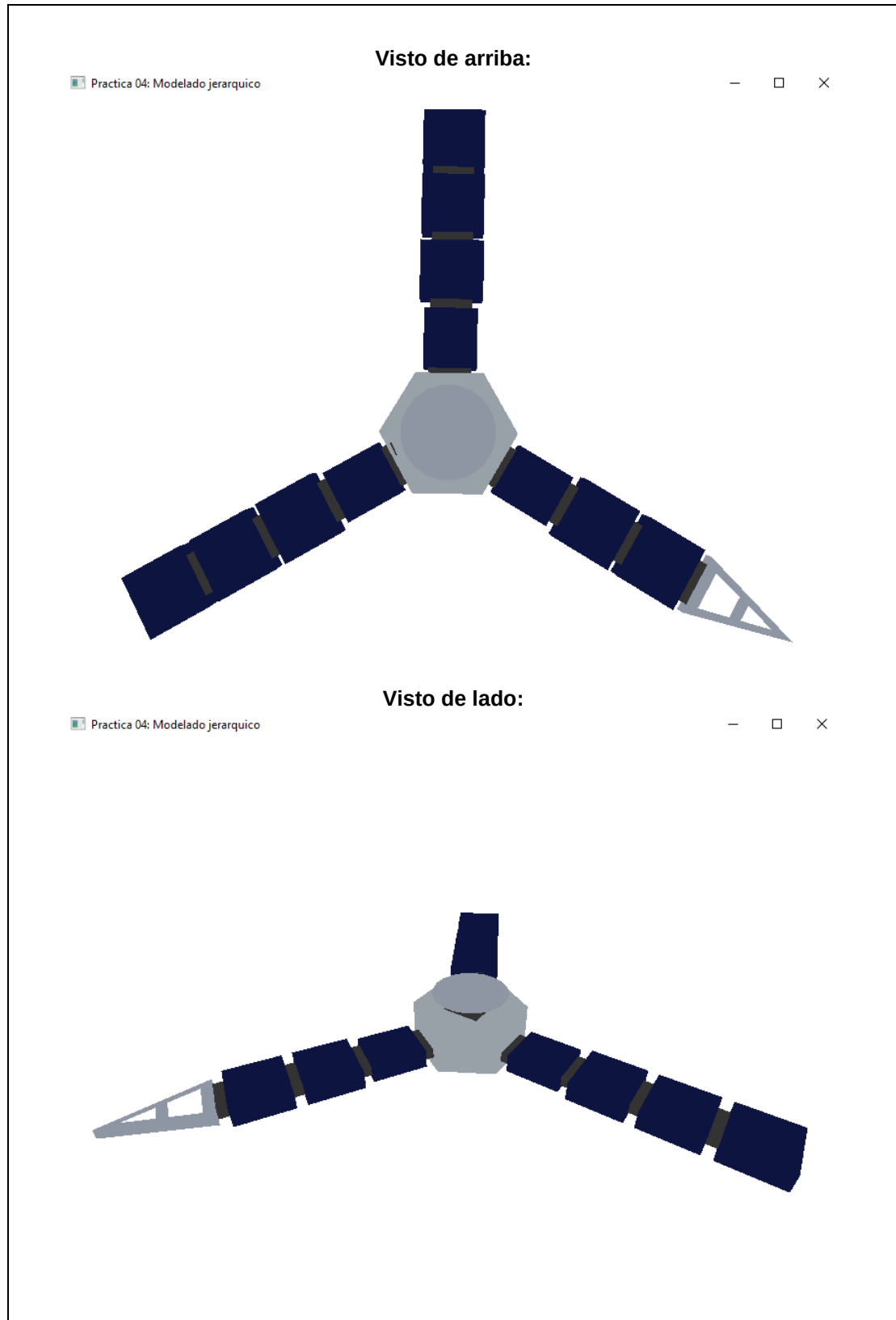
141 /*if (key == GLFW_KEY_E)
142 {
143     theWindow->rotax += 10.0;
144 }
145 if (key == GLFW_KEY_R)
146 {
147     theWindow->rotay += 10.0; //rotar sobre el eje y 10 grados
148 }
149 if (key == GLFW_KEY_T)
150 {
151     theWindow->rotaz += 10.0;
152 }
153 */
```

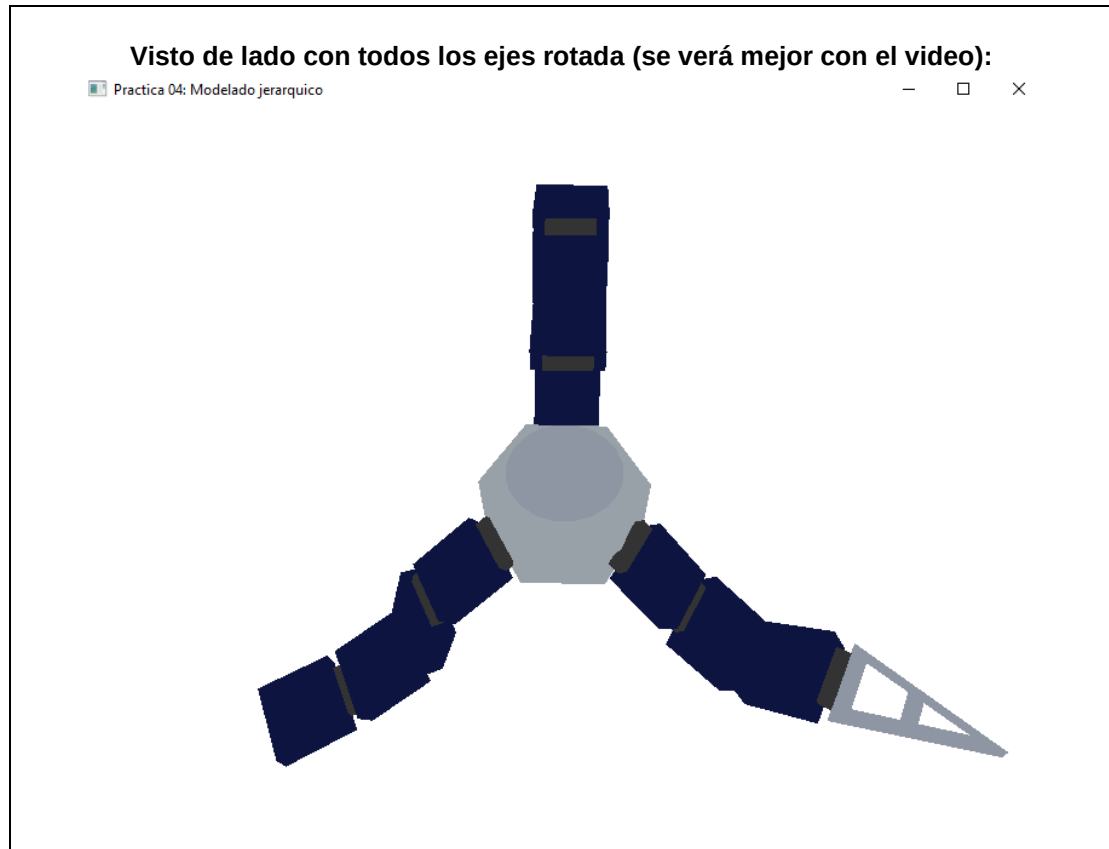
```
219     if (key == GLFW_KEY_C)
220     {
221         theWindow->articulacion17 += 10.0;
222     }
223     if (key == GLFW_KEY_V)
224     {
225         theWindow->articulacion18 += 10.0;
226     }
227     if (key == GLFW_KEY_B)
228     {
229         theWindow->articulacion19 += 10.0;
230     }
231     if (key == GLFW_KEY_N)
232     {
233         theWindow->articulacion20 += 10.0;
234     }
235     if (key == GLFW_KEY_M)
236     {
237         theWindow->articulacion21 += 10.0;
238     }
239     if (key == GLFW_KEY_1)
240     {
241         theWindow->articulacion22 += 10.0;
242     }
243     if (key == GLFW_KEY_2)
244     {
245         theWindow->articulacion23 += 10.0;
246     }
247     if (key == GLFW_KEY_3)
248     {
249         theWindow->articulacion24 += 10.0;
250     }
251     if (key == GLFW_KEY_4)
252     {
253         theWindow->articulacion25 += 10.0;
254     }
255     if (key == GLFW_KEY_5)
256     {
257         theWindow->articulacion26 += 10.0;
258     }
259     if (key == GLFW_KEY_6)
260     {
261         theWindow->articulacion27 += 10.0;
262     }
263     if (key == GLFW_KEY_7)
264     {
265         theWindow->articulacion28 += 10.0;
266     }
267     if (key == GLFW_KEY_8)
268     {
269         theWindow->articulacion29 += 10.0;
270     }
271     if (key == GLFW_KEY_9)
272     {
273         theWindow->articulacion30 += 10.0;
274     }
275 }
```

Finalmente, en *Window.h*, modifiqué los siguientes bloques de código:

```
40     GLfloat getarticulacion18() { return articulacion18; }
41     GLfloat getarticulacion19() { return articulacion19; }
42     GLfloat getarticulacion20() { return articulacion20; }
43     GLfloat getarticulacion21() { return articulacion21; }
44     GLfloat getarticulacion22() { return articulacion22; }
45     GLfloat getarticulacion23() { return articulacion23; }
46     GLfloat getarticulacion24() { return articulacion24; }
47     GLfloat getarticulacion25() { return articulacion25; }
48     GLfloat getarticulacion26() { return articulacion26; }
49     GLfloat getarticulacion27() { return articulacion27; }
50     GLfloat getarticulacion28() { return articulacion28; }
51     GLfloat getarticulacion29() { return articulacion29; }
52     GLfloat getarticulacion30() { return articulacion30; }
53
54 private:
55     GLFWwindow *mainWindow;
56     GLint width, height;
57     GLfloat rotax, rotay, rotaz, articulacion1, articulacion2, articulacion3,
58         articulacion4, articulacion5, articulacion6, articulacion7,
59         articulacion8, articulacion9, articulacion10, articulacion11,
60         articulacion12, articulacion13, articulacion14, articulacion15,
61         articulacion16, articulacion17, articulacion18, articulacion19,
62         articulacion20, articulacion21, articulacion22, articulacion23,
63         articulacion24, articulacion25, articulacion26, articulacion27,
64         articulacion28, articulacion29, articulacion30;
```

Adjunto capturas de pantalla de mi sonda espacial:





2. Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

Para esta práctica no se presentaron problemas.

3. **Conclusión:**

- a. **Los ejercicios del reporte:** Me gustó mucho que ya podemos mover ciertas partes de un conjunto de figuras con una buena implementación jerárquica, y seguir con el tema “espacial” me entusiasma aún más para la realización del proyecto final.
- b. **Comentarios generales:** Considero que fueron ejercicios al nivel de lo que se enseñó en el laboratorio y sirvió para reforzar lo relacionado al modelado jerárquico.
- c. **Conclusión.** Esta práctica me sirvió mucho para tener un mejor entendimiento sobre cómo funcionan las jerarquías en OpenGL, dónde si no hacemos correctamente el guardado de un *model* en un *modelaux*. Nuestra figura tendrá un funcionamiento totalmente distinto al que esperamos. Por ejemplo, para la sonada espacial, tuve que crear dos *modelaux* para poder rotar toda la sonda y a su vez, mantener la jerarquía

en los paneles solares y rotarlos con sus ejes correspondientes. Finalmente, el seguir adentrándonos en la creación de objetos relacionados con el “espacio” me ayuda a darme una mejor idea de cómo crear ciertos elementos para el proyecto final.

Referencias bibliográficas.

Jet Propulsion Laboratory. (2011, May 27). *Juno solar panels complete testing*. NASA. <https://www.jpl.nasa.gov/news/juno-solar-panels-complete-testing/>

BaronDaler. (2019, 29 diciembre). *Sonda espacial Juno modelo 3d*. Free3D. <https://free3d.com/es/modelo-3d/juno-space-probe-4862.html>